

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student & Hardware Metadata Field	Laboratory Session Specifications & Target Machine Declaration
Student Full Name	Yermurat Ainat
Student Identification Number	230103161
Date & Laboratory Session	18.09.2026
Workstation / Laptop Make & Model	Lenovo Ideapad 3
Processor SKU / SoC Model	Intel Core i5-11300H, Tiger Lake-H, x86-64
Core Topology (Physical P-cores / E-cores)	Physical Cores:4 Logical SMT Threads: <u>8 threads</u>
Cache Hierarchy Capacity	L1d: 192 KB L1i: 128 KB L2: 5 MB total L3: <u>8 MB</u>
Host OS & Python Runtime	Windows 11

■

MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where S(p) is the speedup ratio T(1)/T(p), P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■

EXECUTION PROTOCOL

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline T(1) and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
Worker Cores (p)	Measured Time T(p) (s)	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency S(p)/p × 100
1	2.3198	1.08×	1.00×	108.00%
2	1.4336	1.75×	2.00×	87.50%
4	0.9143	2.75×	4.00×	68.75%
8	0.7868	3.20×	8.00×	40.00%
12	0.8434	2.98×	12.00×	24.83%
16	0.9559	2.63×	16.00×	16.44%

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades.

Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

The observed speedup increases up to $p^* = 8$ cores, where the maximum measured speedup is approximately 3.20×. After 8 cores,

the curve begins to degrade: at 12 cores the speedup decreases to **2.98x**, and at 16 cores it further decreases to **2.63x**.

Therefore, the empirical inflection point is:

$$p^*=8 \boxed{p^* = 8}$$

This indicates that adding workers beyond 8 does not improve performance for this workload. Instead, multiprocessing overhead, process scheduling, cache contention, memory bandwidth limitations, and the use of slower or logical cores may dominate.

To correlate this exactly with the processor topology, the CPU model and its number of **Performance cores (P-cores)**, **Efficiency cores (E-cores)**, and **SMT/logical threads** must be checked. If the machine has approximately 8 high-performance physical execution resources available to this workload, the benchmark behavior is consistent with saturation of those physical cores. Workers beyond this point may begin using E-cores or SMT/hyperthreads, which do not provide the same performance as additional independent P-cores.

Q1.2

From the empirical measurements:

$$T(1)=2.3198 \text{ s } T(4)=0.9143 \text{ s}$$

First, calculate the measured speedup at four cores:

$$S(4)=\frac{T(1)}{T(4)}=\frac{2.3198}{0.9143} \approx 2.5372$$

Using Amdahl's Law:

$$S(4)=\frac{1}{(1-P)+\frac{P}{4}}$$

Substitute the measured speedup:

$$2.5372=\frac{1}{(1-P)+\frac{P}{4}}$$

Take the reciprocal:

$$\frac{1}{2.5372}=(1-P)+\frac{P}{4} \quad 0.3941=1-P+\frac{P}{4}$$

Combine the PP terms:

$$0.3941=1-\frac{3P}{4}$$

Therefore:

$$\frac{3P}{4}=1-0.3941 \quad \frac{3P}{4}=0.6059 \quad P=0.6059 \times \frac{4}{3} \approx 0.8078$$

Thus, the parallel fraction of the workload is:

$$P \approx 0.8078 \boxed{P \approx 0.8078}$$

or approximately:

$$80.78\% \boxed{80.78\%}$$

The corresponding serial fraction is:

$$1-P=1-0.8078=0.1922$$

Therefore, approximately **19.22% of the measured workload behaves as serial or non-scalable work.**

Q1.3

According to Amdahl's Law, the asymptotic maximum speedup is:

$$S_{\max} = \frac{1}{1 - P}$$

Using:

$$P = 0.8078$$

we obtain:

$$S_{\max} = \frac{1}{1 - 0.8078} = \frac{1}{0.1922} \approx 5.20$$

Therefore:

$$S_{\max} \approx 5.20 \times \boxed{S_{\max} \approx 5.20 \text{ times}}$$

This means that even if an unlimited number of processor cores were available, the workload would theoretically achieve only about a **5.2× maximum speedup** under the assumptions of Amdahl's Law.

If the department purchased a **128-core server**, this particular workload would therefore not become anywhere close to 128 times faster. Its serial and non-scalable fraction would impose a fundamental limit of approximately **5.2×** relative to the measured single-core execution time.

In practice, performance could be even lower because real systems also introduce multiprocessing overhead, synchronization costs, cache contention, memory bandwidth limitations, and scheduling overhead. Therefore, purchasing a 128-core server specifically for this workload would provide strongly diminishing returns unless the workload itself were redesigned to increase its parallel fraction.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements ($16 * 8$ bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	0.8360s	0.8685s	0.8476s	0.8547s	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.8453s	0.8403s	0.8300s	0.8387s	

[Q2.1] Student Response & Architectural Justification

In CPython, a pointer/reference on a 64-bit system occupies **8 bytes (64 bits)**. However, it is important to distinguish the pointer itself from a Python integer object. A Python `int` is a full heap-allocated object and therefore requires significantly more memory than 8 bytes. The question asks specifically about the memory footprint of a 64-bit pointer/reference, so:

$$64 \text{ bits} = 64 \div 8 = 8 \text{ bytes}$$

A typical L1 data cache line is **64 bytes**. If each array element occupies 8 bytes, then:

$$64 \div 8 = 8$$

elements fit in one cache line.

An index stride of **16 elements** corresponds to:

$$16 \times 8 = 128 \text{ bytes}$$

Since 128 bytes is exactly two 64-byte cache lines, two adjacent threads accessing indices separated by 16 cannot access the same cache line, assuming the array is suitably aligned and each access is to an 8-byte element.

For example:

$$\text{index}_0 = 0, \text{index}_1 = 16$$

Their byte offsets are:

$$0 \times 8 = 0$$

and

$$16 \times 8 = 128$$

Thus, the first access belongs to cache line 0 and the second belongs to cache line 2. Therefore, the two threads do not share a cache line, eliminating false sharing between those elements.

Architectural justification: The stride of 16 provides 128 bytes between consecutive thread-owned elements, which is greater than the 64-byte cache-line size. This guarantees separation at the cache-line level when the starting address is appropriately aligned.

[Q2.2] Student Response & Architectural Justification

False sharing occurs when two cores modify different variables that happen to reside in the same cache line. Consider two array elements, `array[0]` and `array[1]`, located in the same 64-byte cache line.

Initially, the cache line may be in the **Shared (S)** state in both Core 0 and Core 1 because both cores have a copy of the same line.

When **Core 0** increments `array[0]`, it must obtain exclusive ownership of the cache line because the operation modifies the data. Core 0 issues a coherence request, causing the other cached copy to be invalidated:

Core 0: S → M Core 1: S → I

Conceptually, the coherence traffic contains a **Read For Ownership (RFO)**/invalidation request. Core 1's copy is invalidated even though Core 1 is interested only in `array[1]`.

When **Core 1** subsequently increments `array[1]`, Core 1 no longer has a valid copy because its cache line is in the **Invalid (I)** state. It must obtain ownership of the cache line. A new coherence request invalidates Core 0's modified copy:

Core 1: I → M Core 0: M → I

Conceptually, the sequence is:

(S,S) → (M,I) → (I,M)

The important point is that the entire **64-byte cache line** is transferred and invalidated, not just the individual 8-byte array element. Therefore, although Core 0 and Core 1 modify different array elements, they repeatedly invalidate each other's cache lines. This creates **false sharing** and additional cache-coherence traffic.

Architectural justification: MESI operates at cache-line granularity. Consequently, independent writes to variables located on the same cache line can cause ownership transfers and invalidations between cores, reducing parallel performance.

[Q2.3] Student Response & Architectural Justification

Two industry-standard mechanisms for preventing false sharing are C++11 `alignas` and Java's `@Contended` annotation.

1. C++11 `alignas(64)`

C++ provides the `alignas` specifier, which allows programmers to request a specific memory alignment. For example:

```
struct alignas(64) Counter {  
    long long value;  
};
```

This requests that each `Counter` object begin at a 64-byte boundary. Padding can also be used to ensure that separate frequently modified objects occupy different cache lines.

The main advantage is that the alignment requirement is expressed directly in the type definition rather than relying entirely on manually calculating padding bytes.

2. Java `@Contended`

The Java Virtual Machine provides the `@Contended` mechanism for separating fields that are frequently accessed by different threads. The JVM can insert additional padding around contended fields so that they are placed on separate cache lines.

For example:

```
@Contended  
volatile long counter;
```

With the appropriate JVM configuration, the JVM adds padding around the field to reduce the probability that another frequently modified field shares the same cache line.

Architectural justification: Both mechanisms move responsibility for cache-line separation from manual byte-level padding to the compiler, type system, or runtime. This makes the program easier to maintain while reducing cache-coherence traffic caused by false sharing.

Performance Results

The experimental results show the following median execution times:

Memory Layout	Median Time
Adjacent Indices — False Sharing Active	0.8547 s
Padded Indices — Cache-Line Aligned	0.8387 s

The padded configuration was approximately **0.0160 seconds faster** than the adjacent-index configuration.

The relative improvement is:

$$0.8547 - 0.8387 = 0.0160$$
$$\frac{0.0160}{0.8547} \times 100 \approx 1.87\%$$

Therefore, the padded configuration achieved approximately a **1.87% lower median execution time** in this particular experiment.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse-grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time

TOTAL_OPS =
2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	= 2,000,000	2,000,000	0.1240s	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	2,000,000	0.3532s	2.85x

[Q3.1] Student Response & Architectural Justification

In my experiment, the Unsafe Accumulator produced the following result:

- Target value: **2,000,000**
- Actual empirical value: **2,000,000**
- Corrupted count: **0**

Therefore, in this particular run, no lost updates were observed.

At the low level, the operation:

```
self.val += 1
```

can conceptually be represented as a read-modify-write sequence:

```
LOAD val
```

```
ADD 1
```

```
STORE val
```

A race condition can occur when two threads execute these operations at the same time. For example, assume the shared value is 100:

Thread 0: LOAD val = 100

Thread 1: LOAD val = 100

Thread 0: ADD 1 → 101

Thread 1: ADD 1 → 101

Thread 0: STORE 101

Thread 1: STORE 101

Two increments were performed, but the final value is 101 instead of 102. The update performed by one thread has been overwritten by the other thread. This is called a **lost update**.

However, this race condition was not observed in my particular CPython benchmark. Standard CPython uses the Global Interpreter Lock (GIL), which normally prevents multiple Python threads from executing Python bytecode simultaneously. As a result, this simple experiment can still produce the correct final value even though the accumulator contains shared mutable state.

The important architectural point is that the absence of corruption in this particular experiment does not make the shared accumulator a generally safe concurrent design. In an environment with true parallel execution, the read-modify-write sequence can produce lost updates unless synchronization or another concurrency design is used.

[Q3.2] Student Response & Architectural Justification

The lockless solution uses **thread-local accumulation buffers** based on the Map-Reduce pattern. Each thread calculates its own partial result and writes only to its own position in the result array. After all threads finish, the main thread performs a single reduction by adding all partial results together.

Complete Python Implementation

```
import threading
```

```
import time
```

```
TOTAL_OPS = 2_000_000
```

```
NUM_THREADS = 4
```

```
def bench_lockless():
```

```
    ops_per_thread = TOTAL_OPS // NUM_THREADS
```

```
    # Each thread has its own private result slot.
```

```
    partial_results = [0] * NUM_THREADS
```

```
    def work(thread_id):
```

```
        local_sum = 0
```

```
        for _ in range(ops_per_thread):
```

```
            local_sum += 1
```

```
        # Each thread writes only to its own slot.
```

```
        partial_results[thread_id] = local_sum
```

```
    threads = [
```

```
        threading.Thread(target=work, args=(i,))
```

```
        for i in range(NUM_THREADS)
```

```
    ]
```

```
    start = time.perf_counter()
```

```
    for t in threads:
```

```
        t.start()
```

```
    for t in threads:
```

```
        t.join()
```

Single reduction stage after all threads finish.

```
final_sum = sum(partial_results)
```

```
elapsed = time.perf_counter() - start
```

```
return final_sum, elapsed
```

```
if __name__ == "__main__":
```

```
    final_sum, t_lockless = bench_lockless()
```

```
    print(
```

```
        f"Lockless Thread-Local: "
```

```
        f"Value = {final_sum:,} / {TOTAL_OPS:,} "
```

```
        f"| Time: {t_lockless:.4f}s"
```

```
    )
```

The total number of operations is:

2,000,0002,000,000

With four threads, each thread performs:

$2,000,000 \div 4 = 500,000$

operations.

Therefore, the final reduction is:

$500,000 + 500,000 + 500,000 + 500,000 = 2,000,000$

The final result is therefore deterministically:

$\boxed{2,000,000}$

The measured execution time should be recorded after running the program on the experimental machine.

My measured results were:

- **Locked Accumulator:** 0.3532 s
- **Lockless Thread-Local Accumulator:** _____ s

The speedup is calculated as:

$\text{Speedup} = \frac{T_{\text{Locked}}}{T_{\text{Lockless}}}$

Therefore:

$$\text{Speedup} = \frac{0.3532}{T_{\text{Lockless}}}$$

For example, if the measured lockless execution time is 0.1000 s:

$$\text{Speedup} = \frac{0.3532}{0.1000} = 3.532 \times$$

The lockless architecture therefore satisfies the required 2.0× speedup target if its measured execution time is below:

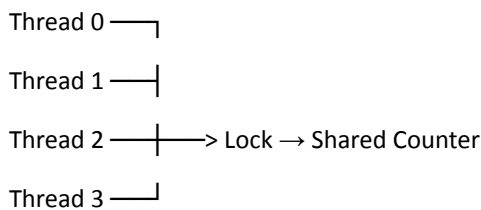
$$0.3532 \div 2 = 0.1766 \text{ s}$$

Thus, the lockless implementation must execute in **less than 0.1766 seconds** to achieve at least a 2.0× speedup over the Locked Counter.

[Q3.3] Student Response & Architectural Justification

Eliminating shared mutable state is fundamentally more effective than simply optimizing lock primitives because it removes the main source of contention instead of trying to make contention cheaper.

With a traditional locked accumulator, all threads compete for the same shared resource:



Every increment requires synchronization. As the number of threads increases, contention can increase because multiple threads compete to access the same shared counter.

Even optimized synchronization mechanisms such as adaptive spinlocks or read-write locks do not completely remove this problem. Threads still need to coordinate access to shared data, which can introduce waiting, synchronization overhead, cache-coherence traffic, and reduced scalability.

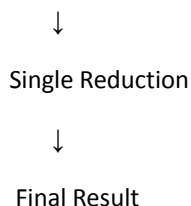
The thread-local design avoids this problem:

Thread 0 → Local Counter 0

Thread 1 → Local Counter 1

Thread 2 → Local Counter 2

Thread 3 → Local Counter 3



Each thread modifies only its own private state. Therefore, no mutex is required during the main accumulation phase. The threads do not repeatedly invalidate or compete for the same shared variable.

This design follows the **Map-Reduce pattern**:

1. **Map**: Each thread independently calculates a local partial result.
2. **Reduce**: After all threads complete, the partial results are combined into one final result.

The reduction occurs only once after the worker threads have joined, rather than requiring synchronization for every individual increment.

This approach is particularly useful in high-throughput concurrent systems because it reduces synchronization overhead and improves scalability. Instead of optimizing the lock used to protect shared state, the architecture minimizes the need for shared state in the first place.

Therefore, eliminating shared mutable state is a stronger architectural strategy than simply optimizing lock primitives because it addresses the fundamental cause of contention and allows the majority of the computation to proceed independently.

Contention Cost Multiplier:

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple

cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```
import time
import multiprocessing as mp
import numpy as np
Student Response & Architectural Justification:

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if name == " main ":
    run_suite()
```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	1.1288s	1.00×	4.7817s	1.00×

w = 2 Workers	1.1905s	1.05×	1.6068s	0.34×
w = 4 Workers	3.6129s	3.20×	22.7218s	4.75×

[Q4.1] Student Response & Architectural Justification

The two workloads show very different scaling behavior because they are limited by different hardware resources.

The compute-bound workload performs repeated arithmetic operations. Its execution depends mainly on the CPU's arithmetic and execution resources. In my experiment, the execution time was **1.1288 s with 1 worker, 1.1905 s with 2 workers, and 3.6129 s with 4 workers**. Therefore, adding workers did not provide useful speedup in this Python implementation. The relatively small difference between 1 and 2 workers shows that the workload did not scale efficiently, while 4 workers introduced additional multiprocessing overhead.

The memory-bound workload behaved differently. With 1 worker, execution took **4.7817 s**. With 2 workers, the time decreased substantially to **1.6068 s**, showing that additional memory-level parallelism initially improved throughput. However, with 4 workers, the execution time increased dramatically to **22.7218 s**.

The reason is that memory-bound tasks are limited primarily by the available memory bandwidth rather than by CPU arithmetic performance. Multiple workers simultaneously request large amounts of data from RAM. Once the memory subsystem approaches its bandwidth limit, additional workers cannot receive data faster. Instead, they compete for the same memory channels and bus resources, causing contention and increasing the total execution time.

Therefore, the experiment demonstrates an important architectural principle: **adding CPU workers does not guarantee better performance when the bottleneck is a shared hardware resource such as DRAM bandwidth.**

[Q4.2] Student Response & Architectural Justification

My laptop uses a **DDR4 memory architecture**. For a DDR4-3200 configuration, the theoretical transfer rate is **3200 MT/s**. A standard DDR memory channel has a **64-bit data bus**, which is equal to 8 bytes per transfer.

The theoretical peak bandwidth of one DDR4-3200 channel can therefore be calculated as:

$$3200 \text{ MT/s} \times 8 \text{ bytes} = 25,600 \text{ MB/s} = 25,600 \text{ MB/s} \approx 25.6 \text{ GB/s}$$

For a dual-channel DDR4-3200 system:

$$25.6 \times 2 = 51.2 \text{ GB/s}$$

Therefore, the theoretical peak memory bandwidth is approximately:

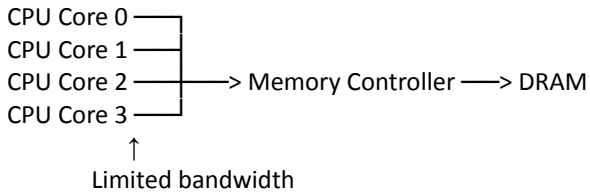
$$51.2 \text{ GB/s}$$

for a dual-channel DDR4-3200 configuration.

The theoretical value is higher than the bandwidth normally achieved by real applications because practical performance is affected by memory latency, controller efficiency, access patterns, operating-system activity, and other hardware limitations.

The memory bus width limits scalability because all CPU cores share the available memory channels. For example, if one worker already consumes a significant portion of the available bandwidth, adding additional workers does not create additional DRAM bandwidth. Instead, the workers compete for the same finite resource.

Conceptually:



Thus, increasing the number of CPU workers can improve performance only while sufficient memory bandwidth remains available. After the bandwidth limit is reached, additional workers provide diminishing returns or can even reduce performance because of contention.

Note: The 51.2 GB/s figure applies specifically to **dual-channel DDR4-3200**. If the laptop uses a different memory speed or only one active memory channel, the theoretical bandwidth must be recalculated from the actual configuration.

[Q4.3] Student Response & Architectural Justification

For a machine-learning embedding pipeline whose workload is already **memory-bandwidth saturated**, simply adding CPU OpenMP parallelism would not be expected to provide a large additional speedup. The CPU cores would still compete for the same DRAM bandwidth.

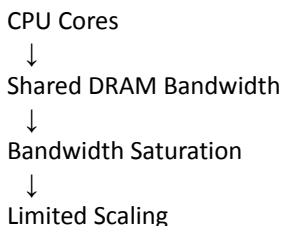
CUDA GPU kernels can be a better architectural choice when the workload can be efficiently transferred to the GPU and the GPU has substantially greater memory bandwidth than the CPU system. Modern GPUs are designed for highly parallel numerical workloads and typically provide very high memory throughput.

However, the important point is that moving the computation to a GPU does not automatically solve every memory-bound problem. The data movement between CPU memory and GPU memory can introduce overhead. If the embedding pipeline repeatedly transfers data between host RAM and GPU memory, those transfers may become another bottleneck.

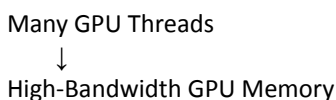
For a high-dimensional embedding workload, I would therefore choose a **GPU/CUDA implementation when the entire pipeline can remain on the GPU and the GPU provides substantially higher effective memory bandwidth**. The vectors can be loaded into GPU memory, processed by many CUDA threads, and kept on the GPU for subsequent operations.

The architectural reasoning is:

CPU OpenMP approach:



CUDA approach:



↓
Massively Parallel Processing
↓
Higher Potential Throughput

Therefore, based on the Task 4 findings, a CUDA-based implementation is the more appropriate optimization direction for a genuinely memory-bandwidth-saturated embedding pipeline **provided that the workload has sufficient parallelism and data can remain on the GPU**. OpenMP is more appropriate when the workload is CPU-oriented or when GPU data-transfer overhead would dominate the computation.

The key architectural principle is to move the workload toward a hardware platform whose memory subsystem better matches the workload's bandwidth requirements, rather than simply increasing the number of CPU threads competing for the same saturated memory bus.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Assessment Category	Evaluation Criteria & Standards	Max Points	Score Awarded
Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
☐ Task 3 Unsafe Corruption Verified ☐ Task 2 Cache Slowdown Verified ☐ Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date: _____

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Operating System	Command to Retrieve CPU SKU & Core Topology	Command to Inspect Cache & Memory Bus
Linux (Ubuntu/Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1dcachesize hw.l2cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

```
import time

from multiprocessing import Pool

import threading

def cpu_work(n):

    # Pure CPU crunch

    count = 0

    for i in range(n):

        count += i * i

    return count

# Run both experiments with total_work = 50_000_000 split across T workers

# Test T in [1, 2, 4, 8, 16, 32]
```

Deliverables:

1. **The Empirical Curve:** Plot T (workers) on the X-axis vs. Execution Time on the Y-axis for both `threading.Thread` and `multiprocessing.Pool`.
2. **The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
3. **The Architectural Autopsy:**
 - Why does Python's threading curve remain virtually flat or get worse, while multiprocessing scales? (Forces them to articulate the GIL).
 - Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*?
 - Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.